1. Explain the difference between structures and unions, What are the practical usage of these in real time applications.

STRUCTURE:

Structure is the user defined data type where we can store the multiple data type as single data type.

Example 1:

```
struct vending_machine {
    int cola;
    char chips;
    double toy;
};
```

- Think of a structure in C as a vending machine with multiple slots.
- Each slot is dedicated to a specific product: one for cola, one for chips, one for a toy, and so on.
- All products are visible simultaneously, each in its designated slot.

In C, a structure operates similarly to this multi-slot vending machine. It can store multiple values of different data types concurrently, with each value occupying its own space.

Example 2:

```
struct student_data {
    char name[20];
    char address[30];
    int age ;
    double height;
};
```

- Student data base.
- Store the student name, address, email, age, height.
- If modifying any member doesn't affect other members.

UNION:

Union similar to the structure, is the user defined data type where we can store the multiple data type as single data type.

Example 1:

- The member in the union shares the same memory location, we can access only one variable at a time.

Sensor Data Interpretation

using unions in C for sensor data interpretation is an excellent example of their practical application, especially in embedded systems where memory efficiency is crucial. Sensors often provide data in raw form, representing different types of measurements. A union can be used to interpret this raw data in various ways.

```
union sensors {
   int temperature; // Temperature data in Celsius
   float pressure;   // Pressure data in hPa
   double humidity;   // Humidity data in percentage
} ;
```

Unions are used in file formats and protocols where a single field can have multiple interpretations depending on the context.

In operating systems, unions can be used to represent system calls that can return different types of data.

2. What are function pointers? Write an example to show the working. Give some real time situations we need to use function pointers?

Function pointer:

- Function pointer are the pointers that points to the function.
- Function pointer are used to store the address of function and we can call that function through pointer by pass as the arguments.

Simple example:

```
void A(){
  printf("I am function A\n");
}

 // callback function
void B(void (*ptr)())
{
    (*ptr)(); // callback to A
}

 int main()
{
```

```
  void (*ptr)() = &A;
 // calling function B and passing
   // address of the function A as argument
    B(ptr);

    return 0;
}
```

Real time examples:

- Interrupt handling in embedded systems, function pointers are used to register callback functions that are called when a particular event occurs.
- Function pointers are used to pass custom comparison functions to sorting and searching algorithms, allowing flexibility in sorting or searching criteria.
- Function pointers are used in Linux-internals signal system call we pass just address of signal handler it's call by OS(operating system) ,ex : signal(SIGINT ,signal_handler);
- Function pointers are also used in Linux-internals threads concept we pass just address of threads function it's call by OS(operating system).

3. What are inline functions? Difference between inline and macros. Inline function?

- The main difference between inline and macro functions is that inline functions are parsed by the compiler, whereas macros in a program are expanded by the preprocessor. The keyword "inline" is used to define an inline function, whereas "#define" is used to define a macro.
- **When should I use Inline Functions over Macros?**
  Use Inline Functions when you need the benefits of function call optimization without sacrificing type safety and debugging capabilities. Inline Functions are suitable for more complex logic and situations where code readability and maintainability are important considerations.
- Inline functions are compiled and also check the compiler errors.
- Macros are pre-processor does not check any compile time errors
- Inline functions are those functions whose definition is small and can be substituted at the place where its function call is made. Basically, they are inlined with its function call. Even, there is no guarantee that the function will actually be inlined. The compiler interprets the inline keyword as a mere hint or requests to substitute the code of the function into its function call.
- Macros have the characteristics for type insensitive and no scope behaviour

4. Explain static keyword with its all possible usage in an application.

- Static keyword is used for local variables , function , global variable , thread local variable.
    - If the variable is declare with the static keyword , it will have the scope accessibility within the block , lifetime become end of the program.
    - It will store in the data segment of the memory , initialize done at only once throughout the program.
- If the static keyword is mentioned before the function , then it has the file scope , that if we try to compile multiple file , function with static keyword have the fie scope , it cannot access by the other file.
- It is used for the encapsulating the data from one file to another file.
- The global static keyword ,having the limitation that the variable has the file scope.
- It doesn't having any linkage property to another file variable.
- Each thread has its own instance of the static local variable, and changes to the variable in one thread do not affect its value in other threads.

5. What are multilevel pointers. Explain its usage with examples?

- Multilevel pointers are used to store the address of pointer variable.
- These pointer are used in DS (data structure) to pass the head value if you want to modify the head value, we have to pass the address of head and collect it through the double pointer.
- If you want allocate the memory for array as both dynamically, at that time we need to use multilevel pointer to store the 2D array.
- And also used in first static second dynamic 2d array memory allocation.

6. What is structure padding. Explain its usage with examples?

- Structure padding is the addition of some empty bytes of memory in the structure to naturally align the data members in the memory.
- It is done to minimize the CPU read cycles to retrieve different data members in the structure.
- It is done helpful for CPU access the data to reduce the instruction cycle.

7. What is volatile. Explain its usage with examples?

The volatile keyword is intended to prevent the compiler from applying any optimizations on objects that can change in ways that cannot be determined by the compiler.

- Global variables modified by an interrupt service routine outside the scope: For example, a global variable can represent a data port (usually a global pointer, referred to as memory mapped IO) which will be updated dynamically. The code reading the data port must be declared as volatile in order to fetch the latest data available at the port. Failing to declare the variable as volatile will result in the compiler optimizing the code in such a way that it will read the port only once and keep using the same value in a temporary register to speed up the program (speed optimization). In general, an ISR is used to update these data ports when there is an interrupt due to the availability of new data.

- Global variables within a multi-threaded application: There are multiple ways for threads' communication, viz., message passing, shared memory, mail boxes, etc. A global variable is weak form of shared memory. When two threads are sharing information via global variables, those variables need to be qualified with volatile. Since threads run asynchronously, any update of global variables due to one thread should be fetched freshly by the other consumer thread. The compiler can read the global variables and place them in temporary variables of the current thread context. To nullify the effect of compiler optimizations, such global variables need to be qualified as volatile.

Can we use const volatile keyword?

Let us declare the const object as volatile and compile code with optimization option. Although we compile code with optimization option, the value of the const object will change because the variable is declared as volatile which means, don't do any optimization.

```
#include <stdio.h>

int main(void)

{

   const volatile int local = 10;

   int *ptr = (int*) &local;


   printf("Initial value of local : %d \n", local);


   *ptr = 100;


   printf("Modified value of local: %d \n", local);


   return 0;

}
```

- The above example may not be a good practical example, but the purpose was to explain how compilers interpret volatile keyword. As a practical example, think of the touch sensor on mobile phones. The driver abstracting the touch sensor will read the location of touch and send it to higher level applications. The driver itself should not modify (const-ness) the read location, and make sure it reads the touch input every time fresh (volatile-ness). Such driver must read the touch sensor input in const volatile manner.

Note: The above codes are compiler specific and may not work on all compilers. The purpose of the examples is to make readers understand the concept.

8. What is void pointer. Explain its usage with examples?

- A void pointer is a pointer that has no associated data type with it. A void pointer can hold an address of any type and can be typecasted to any type.
- void pointers in C are used to implement generic functions in C. For example, compare function which is used in qsort().
- void pointers used along with Function pointers of type void (*)(void) point to the functions that take any arguments and return any value.
- void pointers are mainly used in the implementation of data structures such as linked lists, trees, and queues i.e. dynamic data structures.
- void pointers are also commonly used for typecasting.
- Void pointers are use file handling read and write the data.
- Void pointers are used in Linux-internals threads concept.